USC
UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Escola Técnica Superior de Enxeñaría (ETSE)

Sistemas Operativos II

Grado en Ingeniería Informática

Informe Técnico: Apartado 2

Sincronización de hilos mediante Mutex y Variables de Condición

Autores: [Alan Barreiro Martínez - Adriana Sánchez-Bravo Cuesta]
Asignatura: Sistemas Operativos II
Fecha: 20 de abril de 2026

Índice

1. Introducción **2**

2. Guía de Ejecución **2**

3. Arquitectura del Sistema **2**
3.1. Estructura del Buffer FIFO 2
3.2. Mecanismos de Sincronización 2

4. Análisis del Funcionamiento **3**
4.1. Protocolo de Producción 3
4.2. Protocolo de Consumo 3

5. Resultados y Conclusiones **3**

1. Introducción

El objetivo de este apartado es resolver el problema del productor-consumidor utilizando el estándar de hilos POSIX (*pthreads*). A diferencia de la Práctica 2, donde se emplearon procesos independientes y memoria compartida mapeada (`mmap`), en este ejercicio aprovechamos que los hilos comparten el espacio de direccionamiento virtual dentro del mismo proceso.

El foco principal de este informe técnico es la correcta sustitución de semáforos por **Mutexes** (para garantizar la exclusión mutua) y **Variables de Condición** (para la sincronización de estados, gestionando el bloqueo en buffer lleno o vacío).

2. Guía de Ejecución

El código ha sido implementado en C utilizando la librería `pthread`. Para una correcta compilación y prueba, siga estos pasos:

Terminal

```
1. Compilación del programa (usar flag -lpthread)
gcc -Wall -o programa2 ejerciciodos.c -lpthread
2. Ejecución pasando el fichero de datos como argumento
./ejerciciodos archivoTexto.txt
```

3. Arquitectura del Sistema

3.1. Estructura del Buffer FIFO

El buffer se ha implementado como una estructura circular de enteros (`int`), permitiendo almacenar valores de dos cifras (1-99). Se utiliza una lógica FIFO (*First In First Out*) para asegurar el orden de procesamiento:

```c
typedef struct {
    int buffer[N];     // N definido como tamano del buffer
    int head;          // Indice de lectura (consumidor)
    int tail;          // Indice de escritura (productor)
    int count;         // Contador de elementos presentes
    pthread_mutex_t mutex;
    pthread_cond_t no_lleno;
    pthread_cond_t no_vacio;
} shared_buffer_t;
```

3.2. Mecanismos de Sincronización

Se han empleado las herramientas de sincronización nativas de *pthreads*:

1. **pthread_mutex_t**: Bloquea el acceso al buffer para que solo un hilo opere a la vez, evitando inconsistencias en los índices y el contador.

2. **pthread_cond_t no_vacio**: Permite que el hilo consumidor "duerma"mientras no haya datos, despertando cuando el productor inserte un elemento.

3. **pthread_cond_t no_lleno**: Bloquea al productor si el buffer llega a su capacidad máxima (N), esperando a que el consumidor libere espacio.

4. Análisis del Funcionamiento

4.1. Protocolo de Producción

El hilo productor lee un número del archivo de texto y adquiere el cerrojo (*mutex*). Si detecta que el buffer está saturado (`count == N`), se suspende automáticamente mediante `pthread_cond_wait`. Al despertar, el bucle `while` re-evalúa la condición para garantizar que realmente hay espacio disponible antes de insertar.

4.2. Protocolo de Consumo

El consumidor extrae el entero del índice `head`. Una vez actualizado el contador, emite una señal (`pthread_cond_signal`) a la variable `no_lleno`. Esto señaliza a cualquier hilo productor bloqueado que puede reanudar su operación.

5. Resultados y Conclusiones

Tras las pruebas realizadas, se extraen las siguientes conclusiones técnicas:

1. **Fiabilidad de los datos:** Al final de la ejecución, la suma de los números leídos siempre coincide con lo esperado. Esto demuestra que la exclusión mutua (el Mutex) impide que los hilos se 'pisen' entre ellos.

2. **Mejora en el rendimiento:** A diferencia de otras soluciones donde el programa gasta CPU preguntando sin parar si hay datos, aquí los hilos 'duermen' (bloqueo) gracias a las variables de condición. Esto hace que el programa consuma muy pocos recursos del ordenador mientras espera.

3. **Simplicidad:** Trabajar con hilos y mutexes es más sencillo que manejar procesos separados, ya que la comunicación es directa y el código es mucho más fácil de leer y mantener.